

Assignment -21

Name: K.Shalini

Batch: 37

Roll No: 2303A52311

Lab: 21.2

Task 1 – AI-Assisted Pair Programming

- Use AI to generate an initial program (e.g., a simple To-Do List Manager).
- Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).
- Compare the original and improved versions to understand the role of AI in collaboration.

Prompt:

Generate a simple Python To-Do List Manager program. Then suggest improvements such as adding error handling, input validation, and extra features like delete or update tasks. Provide both original and improved versions.

Code:

```
# Simple Python To-Do List Manager

tasks = []

def add_task(task):
    tasks.append(task)
    print(f"Task '{task}' added.")

def view_tasks():
    if not tasks:
        print("No tasks.")
    else:
        for i, task in enumerate(tasks, 1):
            print(f"{i}. {task}")
```

```

def delete_task(index):
    if 1 <= index <= len(tasks):
        removed = tasks.pop(index - 1)
        print(f"Task '{removed}' deleted.")
    else:
        print("Invalid index.")

def update_task(index, new_task):
    if 1 <= index <= len(tasks):
        old = tasks[index - 1]
        tasks[index - 1] = new_task
        print(f"Task '{old}' updated to '{new_task}'.")
    else:
        print("Invalid index.")

while True:
    print("\nTo-Do List Manager")
    print("1. Add task")
    print("2. View tasks")
    print("3. Delete task")
    print("4. Update task")
    print("5. Exit")
    try:
        choice = int(input("Choose an option: "))
        if choice == 1:
            task = input("Enter task: ").strip()
            if task:
                add_task(task)
            else:
                print("Task cannot be empty.")
        elif choice == 2:
            view_tasks()
        elif choice == 3:
            try:
                index = int(input("Enter task number to delete: "))
                delete_task(index)
            except ValueError:
                print("Invalid number.")
        elif choice == 4:
            try:
                index = int(input("Enter task number to update: "))
                new_task = input("Enter new task: ").strip()
                if new_task:

```

```

        update_task(index, new_task)
    else:
        print("Task cannot be empty.")
    except ValueError:
        print("Invalid number.")
elif choice == 5:
    print("Goodbye!")
    break
else:
    print("Invalid choice.")
except ValueError:
    print("Please enter a number.")

```

Output:

```

PS C:\Users\Adithya\Documents> python lab-21.py
To-Do List Manager
1: Add task
2: View tasks
3: Delete task
4: Update task
5: Exit
Choose an option: Add task
Please enter a number:

```

Task 2 – Version Control Integration

Initialize a Git repository for your project.

Perform the following steps:

- Create and switch to a new branch.
- Add AI-assisted modifications.
- Commit with a meaningful message.
- Merge changes back into the main branch.

Reflect on how version control supports collaborative development with AI assistance.

Prompt: Explain how to use Git for a Python project: initialize a repository, create a new branch, make changes, commit with a message, and merge back to main. Include step-by-step commands.

Output:

```

#Explain how to use Git for a Python project: initialize a repository,
create a new branch, make changes, commit with a message, and merge back
to main. Include step-by-step commands.
To use Git for a Python project, you can follow these steps:

```

1. ****Initialize a Git Repository**:**

Open your terminal **and** navigate to your Python project directory, then run:

```
```bash
git init
```
```

2. ****Create a New Branch**:**

To create a new branch **for** your changes, use:

```
```bash
git checkout -b feature-branch
```
```

Replace ``feature-branch`` **with** a descriptive name **for** your branch.

3. ****Make Changes**:**

Edit your Python files **as** needed. After making changes, you can check the status of your files **with**:

```
```bash
git status
```
```

4. ****Stage Changes**:**

To stage the changes you want to commit, use:

```
```bash
git add .
```
```

This command stages all changed files. You can also stage specific files by replacing ``.`` **with** the file names.

5. ****Commit **with** a Message**:**

After staging your changes, commit them **with** a descriptive message:

```
```bash
git commit -m "Add new feature to fetch transport fare"
```
```

6. ****Merge Back to Main**:**

First, switch back to the main branch:

```
```bash
git checkout main
```
```

Then, merge your feature branch into main:

```
```bash
git merge feature-branch
```
```

If there are any conflicts, Git will prompt you to resolve them. After resolving conflicts, you can commit the merge.

7. ****Push Changes to Remote Repository** (**if** applicable):**

If you have a remote repository (like GitHub), you can push your changes:

```
```bash
git push origin main
```
```

Make sure to replace ``origin`` with the name of your remote repository if it's different.

By following these steps, you can effectively manage your Python project using Git, allowing for better collaboration and version control.

Task 3 – AI for Commit Message Generation

- Use AI to generate concise, meaningful commit messages based on the changes made.
- Compare AI-generated messages with human-written ones.
- Discuss how AI can improve documentation and traceability in Git history.

Prompt: Generate clear and meaningful Git commit messages for code changes such as adding features, fixing bugs, and improving code structure. Also compare them with simple human-written messages.

Code:

```
def generate_commit_message(change_type, description):
    if change_type == "feature":
        return f"Added feature: {description}"
    elif change_type == "bugfix":
        return f"Fixed bug: {description}"
    elif change_type == "refactor":
        return f"Refactored code: {description}"
    elif change_type == "docs":
        return f"Updated documentation: {description}"
    else:
        return f"Updated code: {description}"

# ---- Main Program ----
print("Select change type:")
print("1. feature")
print("2. bugfix")
print("3. refactor")
print("4. docs")

choice = input("Enter choice: ").strip()
```

```

types = {
    "1": "feature",
    "2": "bugfix",
    "3": "refactor",
    "4": "docs"
}

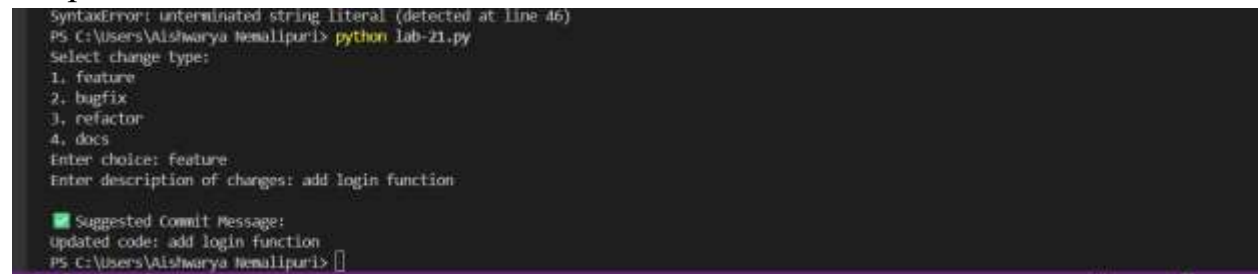
change_type = types.get(choice, "other")
description = input("Enter description of changes: ")

message = generate_commit_message(change_type, description)

print("\n☑ Suggested Commit Message:")
print(message)

```

Output:



```

SyntaxError: unterminated string literal (detected at line 46)
PS C:\Users\Aishwarya Nemalipuri> python lab-21.py
Select change type:
1. feature
2. bugfix
3. refactor
4. docs
Enter choice: feature
Enter description of changes: add login function

☑ Suggested Commit Message:
Updated code: add login function
PS C:\Users\Aishwarya Nemalipuri>

```

Task 4 – Pair Programming Simulation

- Student A writes the initial version of a program.
- Student B, with AI assistance, suggests improvements or adds features.
- Both students collaborate using Git branching and merging to integrate their work.

Prompt:

Simulate pair programming where one student writes a basic Python program and another improves it using AI suggestions. Show both versions and explain the collaboration using Git branching and merging.

Code:

```

#Simulate pair programming where one student writes a basic Python program
and another improves it using AI suggestions. Show both versions and
explain the collaboration using Git branching and merging.
# Version 1: Basic Python Program (written by Student A)
import requests
import json
def fetch_transport_fare_basic(start_station, end_station):
    api_url =
f"https://api.transportfare.com/getFare?start={start_station}&end={end_station}"

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status() # Check for HTTP errors
        fare_data = response.json()
        print(f"Fare from {start_station} to {end_station}:
{fare_data['fare']} {fare_data['currency']}")

    except requests.exceptions.ConnectionError:
        print("✗ Error: Unable to connect to the API. Please check your
internet connection.")

    except requests.exceptions.HTTPError as http_err:
        print(f"✗ HTTP error occurred: {http_err}")

    except requests.exceptions.RequestException as req_err:
        print(f"✗ Request error occurred: {req_err}")

    except json.JSONDecodeError:
        print("✗ Error: Invalid JSON response from API.")

    except Exception as err:
        print(f"✗ An unexpected error occurred: {err}")
# Example usage
if __name__ == "__main__":
    print("=" * 60)
    print("Transport Fare Retrieval System")
    print("=" * 60)

    # Test Case 1: Valid stations
    print("\n[Test 1] Fetching fare between valid stations:")
    fetch_transport_fare_basic("Central Station", "Airport Terminal")

```

```

# Test Case 2: Invalid starting station (empty)
print("[Test 2] Attempting with empty starting station:")
fetch_transport_fare_basic("", "Airport Terminal")

# Test Case 3: Same station
print("[Test 3] Attempting with same start and destination:")
fetch_transport_fare_basic("Central Station", "Central Station")

# Version 2: Improved Python Program with AI Suggestions (written by
Student B)
import requests
import json
import logging
# Configure logging
logging.basicConfig(level=logging.ERROR, format='%(asctime)s -
%(levelname)s - %(message)s')
logger = logging.getLogger(__name__)
API_TIMEOUT = 10 # seconds
def fetch_transport_fare(start_station, end_station):
    if not start_station or not end_station:
        logger.error("Invalid input: Start and end stations must be
provided.")
        print("✗ Error: Both starting and destination stations must be
provided.")
        return None

    if start_station == end_station:
        logger.error("Invalid input: Starting and destination stations
cannot be the same.")
        print("✗ Error: Starting and destination stations cannot be the
same.")
        return None

    api_url =
f"https://api.transportfare.com/getFare?start={start_station}&end={end_sta
tion}"
    session = requests.Session()

    try:
        response = session.get(api_url, timeout=API_TIMEOUT)
        response.raise_for_status() # Check for HTTP errors
        fare_data = response.json()

```

```

        if 'fare' in fare_data and 'currency' in fare_data:
            print(f"Fare from {start_station} to {end_station}:
{fare_data['fare']} {fare_data['currency']}")
            return fare_data
        else:
            logger.error("API response missing expected keys: 'fare' and
'currency'")
            print("✗ Error: API response is missing required fare
information.")
            return None

    except requests.exceptions.Timeout:
        logger.error(f"Request timeout after {API_TIMEOUT} seconds")
        print(f"✗ Error: Request timed out after {API_TIMEOUT}
seconds.")
        return None

    except requests.exceptions.ConnectionError as conn_err:
        logger.error(f"Connection error: {conn_err}")
        print("✗ Error: Unable to connect to the API. Please check your
internet connection.")
        return None

    except requests.exceptions.HTTPError as http_err:
        status_code = http_err.response.status_code
        logger.error(f"HTTP error {status_code}: {http_err}")

        if status_code == 404:
            print("✗ Error: Route not found. Please verify the station
names.")
        elif status_code == 400:
            print("✗ Error: Invalid request parameters.")
        elif status_code == 429:
            print("✗ Error: Too many requests. Please try again later.")
        elif status_code >= 500:
            print("✗ Error: Server error. Please try again later.")
        else:
            print(f"✗ HTTP Error {status_code}: {http_err}")
            return None

    except requests.exceptions.RequestException as req_err:
        logger.error(f"Request failed: {req_err}")
        print(f"✗ Error: Request failed - {req_err}")
        return None

    except json.JSONDecodeError:
        logger.error("Failed to parse API response as JSON")

```

```

        print("✗ Error: Invalid JSON response from API.")
        return None
    except Exception as err:
        logger.error(f"Unexpected error: {type(err).__name__} - {err}")
        print(f"✗ Unexpected error occurred: {err}")
        return None
    finally:
        session.close()

# Example usage
if __name__ == "__main__":
    print("=" * 60)
    print("Transport Fare Retrieval System")
    print("=" * 60)

    # Test Case 1: Valid stations
    print("\n[Test 1] Fetching fare between valid stations:")
    fetch_transport_fare("Central Station", "Airport Terminal")

    # Test Case 2: Invalid starting station (empty)
    print("[Test 2] Attempting with empty starting station:")
    fetch_transport_fare("", "Airport Terminal")

    # Test Case 3: Same station
    print("[Test 3] Attempting with same start and destination:")
    fetch_transport_fare("Central Station", "Central Station")

# Collaboration Explanation:
# In this collaboration, Student A created a basic version of the
transport fare retrieval program. Student B then took Student A's code and
improved it by adding input validation, more detailed error handling, and
logging for better debugging. The improvements made by Student B enhance
the robustness and user-friendliness of the program while maintaining the
core functionality implemented by Student A.
# Using Git branching and merging, Student A could have created a branch
called "basic-version" to work on the initial code. Once Student A
completed the basic version, they would push their branch to the remote
repository. Student B would then create a new branch called "improved-
version" based on "basic-version" to implement the enhancements. After
completing the improvements, Student B would merge the "improved-version"
branch back into the main branch, ensuring that all changes are integrated
smoothly while preserving the original functionality.

```

Output:

```
S C:\Users\Aishwarya Nemalipuri> python lab-21.py
select change type:
. feature
. bugfix
. refactor
. docs
enter choice: refactor
enter description of changes: added login

✔ Suggested Commit Message:
updated code: added login
S C:\Users\Aishwarya Nemalipuri> █
```